

Q3.1

a):

Code:

```
% Import images and ground truth
images = {'document01.bmp','document02.bmp','document03.bmp','document04.bmp'};
gts = {'document01-GT.tiff','document02-GT.tiff','document03-GT.tiff','document04-GT.tiff'};

results = struct();

%%
for idx = 1:length(images)
    % Load image and GT
    I = imread(images{idx});
    I = rgb2gray(I);
    GT = imread(gts{idx});

    % Apply Otsu global thresholding
    level = graythresh(I);
    BW = imbinarize(I, level);

    % Evaluate difference
    diff_img = abs(double(BW) - double(GT));
    error_sum = sum(diff_img(:));

    % Store results
    results(idx).filename = images{idx};
    results(idx).otsu_threshold = level;
    results(idx).error = error_sum;

    % Display
    figure;
    subplot(1,3,1); imshow(I); title('Original');
    subplot(1,3,2); imshow(BW); title('Otsu Segmentation');
    subplot(1,3,3); imshow(diff_img, []); title('Difference Image');
end

% Show results summary
disp(struct2table(results));
```

Result:



Comments: Otsu segmentation provides good overall contrast between the text and the background however, it struggles to remove stains present in the image. This limitation arises

because Otsu's global thresholding method applies a single threshold value across the entire image, making it unable to adapt to local intensity variations. As a result, certain background artifacts and stains are incorrectly classified, leading to segmentation errors in those regions.

b):

Code:

```
%%
for idx = 1:length(images)
    I = imread(images{idx});
    I = rgb2gray(I);
    GT = imread(gts{idx});
    I = im2double(I);

    % Parameters for Niblack
    windowSize = 300; % change this value
    k = -1; % change this value

    % Compute local mean and standard deviation
    meanI = movmean(I, [windowSize windowSize], 'Endpoints', 'shrink');
    stdI = movstd(I, [windowSize windowSize], 0, 'Endpoints', 'shrink');

    % Niblack threshold
    T = meanI + k * stdI;

    % Binarize image
    BW = I <= T;

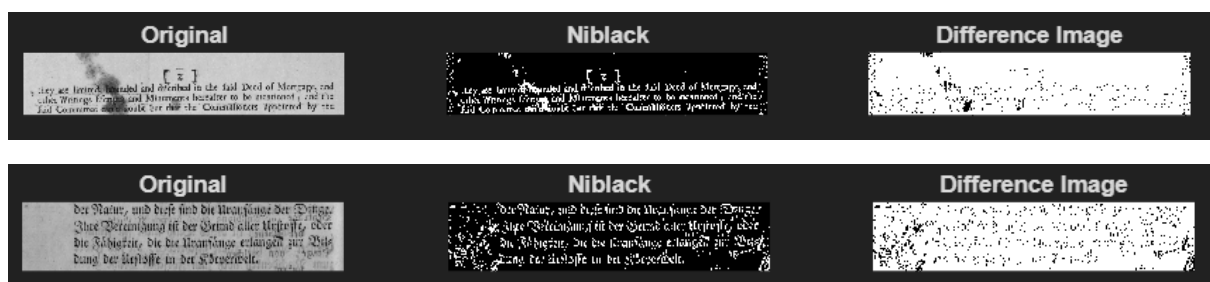
    % Quantitative evaluation
    diff_img = abs(double(BW) - double(GT));
    error_sum = sum(diff_img(:));

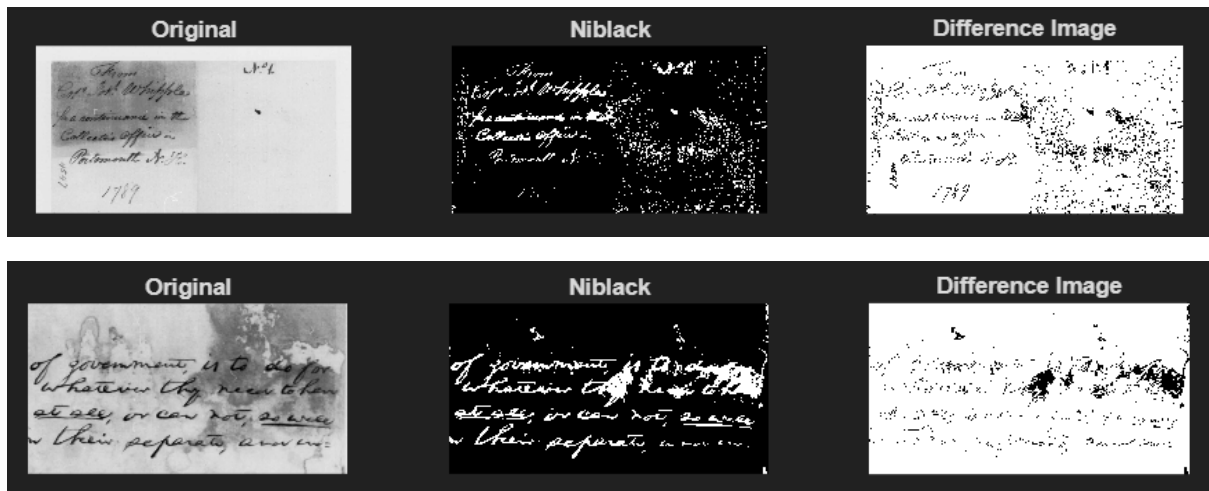
    results(idx).filename = images{idx};
    results(idx).window = windowSize;
    results(idx).k = k;
    results(idx).error = error_sum;

    figure;
    subplot(1,3,1); imshow(I); title('Original');
    subplot(1,3,2); imshow(BW); title(['Niblack']);
    subplot(1,3,3); imshow(diff_img, []); title('Difference Image');
end

disp(struct2table(results));
```

Result:





Comments:

Niblack's local thresholding algorithm adapts the threshold value for each pixel based on the local mean and standard deviation within a window, making it effective for text segmentation in images with uneven lighting or varying background intensities. By tuning the parameter k , which controls the sensitivity of the threshold, different levels of text-background separation can be achieved. A smaller k value tends to include more background noise, while a larger k may result in missing finer text details. The optimal segmentation result is obtained when the chosen k value balances the retention of text strokes and the suppression of background noise, leading to clear, well-defined text regions without excessive fragmentation or merging.

c)

Code:

```
%%
for idx = 1:length(images)
    I = imread(images{idx});
    I = rgb2gray(I);
    GT = imread(gts{idx});
    I = im2double(I);

    % Parameters for Niblack
    windowSize = 300; % change this value
    k = -1; % change this value

    % Compute local mean and standard deviation
    meanI = movmean(I, [windowSize windowSize], 'Endpoints', 'shrink');
    stdI = movstd(I, [windowSize windowSize], 0, 'Endpoints', 'shrink');

    % Niblack threshold
    T = meanI + k * stdI;

    % Binarize image
    BW = I <= T;

    BW_clean = bwareaopen(BW, 70); % Remove small noisy regions
    BW_clean = imclose(BW_clean, strel('disk', 1)); % Fill text gaps

    % Quantitative evaluation
    diff_img = abs(double(BW_clean) - double(GT));
    error_sum = sum(diff_img(:));

    results(idx).filename = images{idx};
    results(idx).window = windowSize;
    results(idx).k = k;
    results(idx).error = error_sum;

    figure;
    subplot(1,3,1); imshow(I); title('Original');
    subplot(1,3,2); imshow(BW_clean); title(['Niblack']);
    subplot(1,3,3); imshow(diff_img, []); title('Difference Image');
end

disp(struct2table(results));
```

Result:



Comments:

I applied morphological cleanup which enhances the quality of the binary output after Niblack thresholding. The use of `bwareaopen()` effectively removes small, isolated noise components that are often misclassified as text due to local threshold fluctuations. Following that, the `imclose()` operation with a small disk structuring element bridges small gaps within text characters and between disconnected strokes, improving the continuity of the segmented text. Together, these steps produce a cleaner and more readable binary image by maintaining the integrity of actual text regions while suppressing non-text noise, thereby improving the visual and quantitative segmentation accuracy.

Q3.2

a)

Code:

```
%%
function D = computeDisparitySSD(P1, Pr, winSize, maxDisp)
    P1 = double(P1);
    Pr = double(Pr);
    [rows, cols] = size(P1);
    halfWin = floor(winSize / 2);
    D = zeros(rows - 2 * halfWin, cols - 2 * halfWin);

    % Loop through each pixel within the valid region
    for i = 1 + halfWin : rows - halfWin
        for j = 1 + halfWin : cols - halfWin
            templateLeft = P1(i - halfWin : i + halfWin, j - halfWin : j + halfWin);
            minSSD = inf;
            bestDisp = 0;

            for d = 0:maxDisp
                jr = j - d;

                if (jr - halfWin < 1)
                    continue;
                end

                templateRight = Pr(i - halfWin : i + halfWin, jr - halfWin : jr + halfWin);
                SSD = sum(sum((templateLeft - templateRight) .^ 2));

                if SSD < minSSD
                    minSSD = SSD;
                    bestDisp = d;
                end
            end

            D(i - halfWin, j - halfWin) = -bestDisp;
        end
    end
end
```

b) + c)

code:

```
%%  
%%  
  
P1 = rgb2gray(im2double(imread('corridorl.jpg')));  
Pr = rgb2gray(im2double(imread('corridorr.jpg')));  
  
D = computeDisparitySSD(P1, Pr, 11, 15);  
  
figure;  
imshow(-D, [-15,15]);  
colormap(gray);  
colorbar;  
title('Disparity Map');
```

Result:



Comments:

In the disparity map, nearer points appear bright while farther points appear dark, reflecting the inverse relationship between disparity and depth. The quality of the computed disparities depends strongly on the local image structure. Areas with rich texture and clear edges produce more reliable disparity estimates because the matching between left and right images is more distinct and less ambiguous. Conversely, regions with low texture, repetitive patterns, or uniform intensity often result in poor or noisy disparity estimates, as multiple locations in the right image may produce similar matching costs. This leads to unstable or incorrect disparity values in those areas.

d)

Code:

```
P1 = im2double(rgb2gray(imread('triclopsl.jpg')));  
Pr = im2double(rgb2gray(imread('triclopsr.jpg')));  
  
D = computeDisparitySSD(P1, Pr, 11, 15);  
  
figure;  
imshow(-D, [-15,15]);  
colormap(gray);  
colorbar;  
title('Disparity Map');
```

Result:



Comments:

The image structure of the stereo pair directly influences the accuracy of the estimated disparities. Textured regions and sharp intensity variations provide distinctive local features that facilitate accurate correspondence matching between the left and right images. In contrast, homogeneous regions lacking texture, such as smooth walls or sky, offer insufficient visual cues for matching, leading to uncertain or erroneous disparity values. Additionally, occlusion boundaries and shadowed areas can further degrade accuracy because corresponding pixels may be absent or significantly different between views. Thus, the disparity estimation performs best in well-textured, high-contrast regions and deteriorates in flat or repetitive image structures.